GA GUARDIAN

LayerZero

Stellar OneSig

Security Assessment

June 24th, 2026

Summary

Audit Firm Guardian

Prepared By CuriousApple, Jakub Zmyslowski

Client Firm LayerZero

Final Report Date June 24, 2026

Audit Summary

LayerZero engaged Guardian to review the Stellar implementation of OneSig. From May 25 through June 3rd, 2026, Guardian reviewed OneSig on Stellar and recorded the findings in this report.

Confidence Ranking

Given the lack of High and Critical issues detected during the main review, Guardian assigns a Confidence Ranking of 5 to the protocol. Guardian advises the protocol to consider periodic review with future changes. For detailed understanding of the Guardian Confidence Ranking, please see the rubric on the following page.

✅ Verify the authenticity of this report on Guardian's GitHub: https://github.com/guardianaudits

Guardian Confidence Ranking

Confidence Ranking	Definition and Recommendation	Risk Profile
5: Very High Confidence	Codebase is mature, clean, and secure. No High or Critical vulnerabilities were found. Follows modern best practices with high test coverage and thoughtful design. **Recommendation:** Code is highly secure at time of audit. Low risk of latent critical issues.	0 High/Critical findings and few Low/Medium severity findings.
4: High Confidence	Code is clean, well-structured, and adheres to best practices. Only 1 Significant issue was uncovered per week. Design patterns are sound, and test coverage is strong. **Recommendation:** Suitable for deployment after remediations; consider periodic review with changes.	0-1 High/Critical findings per engagement week and little to no Medium severity issues. Varied Low severity findings.
3: Moderate Confidence	Medium-severity and occasional High-severity issues found. Code is functional, but there are concerning areas (e.g., weak modularity, risky patterns). No critical design flaws, though some patterns could lead to issues in edge cases. **Recommendation:** Address issues thoroughly and consider a targeted follow-up audit depending on code changes.	1-2 High/Critical findings per engagement week.
2: Low Confidence	Code shows frequent emergence of Critical/High vulnerabilities. Audit revealed recurring anti-patterns, weak test coverage, or unclear logic. These characteristics suggest a high likelihood of latent issues. **Recommendation:** Post-audit development and a second audit cycle are strongly advised.	2-4 High/Critical findings per engagement week. Or additional High/Critical findings uncovered in remediation review which have not been resolved and confirmed by Guardian.
1: Very Low Confidence	Code has systemic issues. Multiple High/Critical findings (≥5/week), poor security posture, and design flaws that introduce compounding risks. Safety cannot be assured. **Recommendation:** Halt deployment and seek a comprehensive re-audit after substantial refactoring.	≥5 High/Critical findings and overall systemic flaws.

Table of Contents

Project Information

Project Overview 5

Audit Scope & Methodology 6

Smart Contract Risk Assessment

Findings & Resolutions 9

Addendum

Disclaimer 17

About Guardian 18

Project Overview

Project Summary

Project Name	Stellar OneSig
Codebase	https://github.com/LayerZero-Labs/monorepo-external
Commit(s)	0781f4e650ec611eae75d538889a66866f52fa70

Audit Summary

Delivery Date	June 24, 2026
Audit Methodology	Static Analysis, Manual Review

Vulnerability Summary

Vulnerability Level	Total	Pending	Declined	Acknowledged	Partially Resolved	Resolved
Critical	0	0	0	0	0	0
High	0	0	0	0	0	0
Medium	0	0	0	0	0	0
Low	4	0	0	4	0	0
Info	3	0	0	3	0	0

Audit Scope & Methodology

Core Contracts

auth.rs (287)
eip712.rs (178)
onesig.rs (108)
executor.rs (95)

Supporting Modules

events.rs
storage.rs
lib.rs
errors.rs

Audit Scope & Methodology

Vulnerability Classifications

Severity	Impact: *High*	Impact: *Medium*	Impact: *Low*
Likelihood: *High*	🔴 Critical	🟠 High	🟡 Medium
Likelihood: *Medium*	🟠 High	🟡 Medium	🟢 Low
Likelihood: *Low*	🟡 Medium	🟢 Low	🟢 Low

Impact

High Significant loss of assets in the protocol, significant harm to a group of users, or a core functionality of the protocol is disrupted.

Medium A small amount of funds can be lost or ancillary functionality of the protocol is affected. The user or protocol may experience reduced or delayed receipt of intended funds.

Low Can lead to any unexpected behavior with some of the protocol's functionalities that is notable but does not meet the criteria for a higher severity.

Likelihood

High The attack is possible with reasonable assumptions that mimic on-chain conditions, and the cost of the attack is relatively low compared to the amount gained or the disruption to the protocol.

Medium An attack vector that is only possible in uncommon cases or requires a large amount of capital to exercise relative to the amount gained or the disruption to the protocol.

Low Unlikely to ever occur in production.

Audit Scope & Methodology

Methodology

Guardian is the ultimate standard for Smart Contract security. An engagement with Guardian entails the following:

- Two competing teams of Guardian security researchers performing an independent review.
- A dedicated fuzzing engineer to construct a comprehensive stateful fuzzing suite for the project.
- An engagement lead security researcher coordinating the 2 teams, performing their own analysis, relaying findings to the client, and orchestrating the testing/verification efforts.

The auditing process pays special attention to the following considerations:

- Testing the smart contracts against both common and uncommon attack vectors.
- Assessing the codebase to ensure compliance with current best practices and industry standards.
- Ensuring contract logic meets the specifications and intentions of the client.
- Cross-referencing contract structure and implementation against similar smart contracts produced by industry leaders.
- Thorough line-by-line manual review of the entire codebase by industry experts. Comprehensive written tests as a part of a code coverage testing suite.
- Contract fuzzing for increased attack resilience.

Findings & Resolutions

ID	Title	Category	Severity	Status
L-01	Secp256k1 Signatures Are Not Restricted To Low-s Form	Best Practices	Low	Acknowledged
L-02	Secp256k1 Recovery IDs 2 And 3 Are Accepted	Best Practices	Low	Acknowledged
L-03	Signer-as-executor Delegate Is Typed As Dynamic Bytes While Stored As BytesN<32>	Best Practices	Low	Acknowledged
L-04	Offchain Leaf Encoding Does Not Enforce Bounds	Validation	Low	Acknowledged
I-01	OneSig Stellar Depends On Stellar Archival Restoration After Inactivity	Warning	Info	Acknowledged
I-02	EIP-712 Expiry Fields Use Different Integer Types Across Signed Structs	Best Practices	Info	Acknowledged
I-03	Signed Sub-invocation Trees Can Become Stale	Warning	Info	Acknowledged

L-01 | secp256k1 Signatures are not Restricted to low-s Form

Category	Severity	Location	Status
Best Practices	● Low	contracts/protocol/stellar/contracts/utils/src/multisig.rs	Acknowledged

Description

OneSig Stellar recovers secp256k1 signers from `(r, s, v)` signatures but does not explicitly reject high-s signatures. This allows multiple valid encodings for the same signer and digest.

This does not appear to bypass threshold checks: the contract counts recovered signer addresses and enforces strictly increasing signer order, so the same signer cannot be counted twice even with malleable signatures.

The remaining risk is canonicalization and ecosystem compatibility. Off-chain tooling that assumes canonical Ethereum signatures may disagree with on-chain acceptance, and signature deduplication by bytes rather than recovered signer can be misleading.

Recommendation

Reject high-s signatures during recovery. Expected rule:

```
    s <= secp256k1n / 2
```

This aligns the Stellar verifier with standard Ethereum ECDSA canonical-signature expectations.

Resolution

LayerZero: Acknowledged.

L-02 | secp256k1 Recovery IDs 2 and 3 are Accepted

Category	Severity	Location	Status
Best Practices	Low	contracts/protocol/stellar/contracts/utils/src/multisig.rs	Acknowledged

Description

The secp256k1 recovery helper accepts Ethereum-style `v` values in the range `27..=30`, normalizing them to recovery IDs `0..=3`. Ethereum signatures conventionally use `v = 27` or `28`, corresponding to recovery IDs `0` and `1`.

Accepting `29` and `30` broadens the accepted signature domain beyond normal Ethereum-style expectations. The code may also accept raw recovery IDs outside the Ethereum-style range if the host recovery function accepts them.

This does not appear to bypass threshold checks, but it can create differences between off-chain Ethereum-compatible validation and on-chain Stellar validation.

Recommendation

Restrict accepted values to one canonical set: `v in {27, 28}` or `v in {0, 1}`. If both formats are supported, explicitly accept only `{0, 1, 27, 28}` and reject all other values.

Resolution

LayerZero: Acknowledged.

L-03 | Signer-as-executor Delegate Uses Dynamic Bytes While Stored as BytesN<32>

Category	Severity	Location	Status
Best Practices	Low	packages/onesig/onesig-stellar/contracts/onesig/src/interfaces/onesig.rs	Acknowledged

Description

The signer-as-executor EIP-712 struct declares `delegate` as dynamic `bytes`, while the Rust contract stores it as `BytesN<32>`. The implementation intentionally hashes the delegate as dynamic bytes, and the repository TypeScript helper matches this behavior.

Independent clients may reasonably encode the field as `bytes32` because the on-chain value is fixed-size. For EIP-712, dynamic `bytes` is encoded as `keccak256(delegate)`, while `bytes32` is encoded directly. These produce different struct hashes and can make signer-as-executor proofs fail due to off-chain/on-chain digest mismatch.

Failures manifest as authorization rejection, not as a threshold bypass.

Recommendation

Either change the EIP-712 type to `bytes32 delegate`, or keep `bytes delegate` and document that the 32-byte Ed25519 delegate key is encoded as dynamic bytes using `keccak256(delegate)`.

If external clients are expected, add cross-language test vectors for signer-as-executor proofs.

Resolution

LayerZero: Acknowledged.

L-04 | Offchain Leaf Encoding Does not Enforce Bounds

Category	Severity	Location	Status
Validation	Low	packages/onesig/onesig-core/src/	Acknowledged

Description

The Stellar OneSig contract hashes `oneSigId` and `nonce` as Rust `u64` values when constructing each leaf. The shared TypeScript encoder accepts these fields as arbitrary `bigint` values, pads them to hex, and effectively reads only the first eight bytes without first checking the `uint64` bounds.

Out-of-range or negative inputs can therefore be encoded non-canonically. Different bigint values can share the same effective on-chain `u64` prefix, while duplicate detection uses the raw `nonce.oneSigId` bigint pair before encoding.

This does not let an attacker forge Merkle proofs or bypass threshold signatures. The risk is off-chain proposal canonicalization: signer tooling can display one `nonce` or `oneSigId` while the generated leaf commits to a different effective `uint64`, bypassing duplicate proposal checks or producing roots that are misleading or fail to execute.

Recommendation

Reject non-canonical values before leaf construction and duplicate detection:

```
    const U64_MAX = (1n << 64n) - 1n;
    if (value < 0n || value > U64_MAX) throw new Error(`${name} must fit uint64`);
```

Apply this validation to both `nonce` and `oneSigId`.

Resolution

LayerZero: Acknowledged.

I-01 | OneSig Stellar Depends On Stellar Archival Restoration after Inactivity

Category	Severity	Location	Status
Warning	• Info	Global	Acknowledged

Description

OneSig Stellar stores critical contract state such as `Nonce`, `Seed`, and `OneSigId` in Soroban instance storage. Instance storage has a ledger-entry TTL. The contract uses LayerZero Stellar macros that initialize default TTL configuration in the constructor and inject instance TTL extension at the start of contract entrypoints.

This means active contracts are maintained by call-triggered TTL extension. However, the extension does not run in the background. If a deployment receives no invocations until its instance TTL reaches zero, the instance entry becomes archived and must be restored before normal execution can proceed.

Impact

This is an operational and liveness dependency, not a direct authorization bypass or replay issue.

When the instance entry is archived:

• OneSig state is not reset to defaults.
• Previously consumed nonces should remain consumed after restoration.
• Execution still requires the normal OneSig authorization path: a valid Merkle root, threshold signatures, current nonce proof, and executor or signer permission when configured.
• Clients or executors that do not handle Stellar archival restoration may fail to execute valid OneSig transactions after long inactivity.

Restoration can be performed independently of OneSig:

• Automatically, when an `InvokeHostFunction` transaction is simulated and the archived entries are included in the restore list.
• Manually, by submitting a Stellar `RestoreFootprintOp` for the archived footprint.

Neither path requires OneSig multisig authorization because restoration is a Stellar ledger operation, not a OneSig business-logic operation.

Recommendation

Document this operational requirement for OneSig Stellar executors and clients:

• Always simulate Soroban invocations and preserve restore data in the assembled transaction.
• Ensure executor tooling can detect simulation restore preambles and retry or submit restoration as needed.
• Optionally run maintenance transactions to extend contract instance TTL before expiry for high-value deployments.

Resolution

LayerZero: Acknowledged.

I-02 | EIP-712 Expiry Fields Use Different Integer Types Across Signed Structs

Category	Severity	Location	Status
Best Practices	• Info	packages/onesig/onesig-stellar/contracts/onesig/src/eip712.rs	Acknowledged

Description

The Merkle-root signature and signer-as-executor proof use different EIP-712 integer types for expiry values:

```
SignMerkleRoot(..., uint256 expiry)
SignerProof(..., uint64 signerProofExpiry)
```

Both values are represented as `u64` in the Stellar contract and are encoded into 32-byte ABI-style words when constructing the digest. The repository's code and tests are internally consistent, so this is not a direct vulnerability.

The issue is an interoperability footgun for independent clients that must exactly match the declared EIP-712 type strings.

Merkle root type:

```
SignMerkleRoot(bytes32 seed,bytes32 merkleRoot,uint256 expiry)
```

Signer proof type:

```
SignerProof(bytes32 leafHash,bytes32 merkleRoot,bytes delegate,uint64 signerProofExpiry)
```

Because EIP-712 type strings are part of the type hash, clients must use the exact declared type. Treating the signer proof expiry as `uint256`, or changing the Merkle-root expiry to `uint64`, will produce a different digest.

Impact

• Independent clients may produce invalid signatures if they normalize both expiry fields to the same type.
• Reviewers may incorrectly assume the two expiry fields share the same typed-data encoding.

Recommendation

Standardize the expiry field type across OneSig typed-data structs if possible.

If the mismatch is intentional, document it and provide test vectors for:

• Merkle-root signing digest
• signer-as-executor proof digest

Resolution

LayerZero: Acknowledged.

I-03 | Signed Sub-invocation Trees Can Become Stale

Category	Severity	Location	Status
Warning	• Info	Global	Acknowledged

Description

OneSig Stellar commits to exact Soroban `sub_invocations` as part of the signed Merkle leaf. This is important for security because it prevents an executor from adding broader nested authorization after signers approve a root. However, it also means signed leaves are sensitive to changes in target-contract authorization behavior between simulation/signing and execution.

If the required Soroban sub-invocation tree changes after signers approve a leaf, the executor cannot update the `sub_invocations` without changing the leaf hash and invalidating the Merkle proof. The old leaf may become unexecutable and block later nonces.

The original signed leaf may fail execution, and because OneSig uses a shared monotonic nonce per deployment, later nonce leaves cannot execute until the current nonce is resolved.

Recommendation

Prefer stable target-contract authorization paths for OneSig-managed operations.

Resolution

LayerZero: Acknowledged.

Disclaimer

This report is not, nor should be considered, an “endorsement” or “disapproval” of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any “product” or “asset” created by any team or project that contracts Guardian to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. Guardian’s position is that each company and individual are responsible for their own due diligence and continuous security. Guardian’s goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies, and in no way claims any guarantee of security or functionality of the technology we agree to analyze.

The assessment services provided by Guardian is subject to dependencies and under continuing development. You agree that your access and/or use, including but not limited to any services, reports, and materials, will be at your sole risk on an as-is, where-is, and as-available basis. Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives, and other unpredictable results. The services may access, and depend upon, multiple layers of third-parties.

Notice that smart contracts deployed on the blockchain are not resistant from internal/external exploit. Notice that active smart contract owner privileges constitute an elevated impact to any smart contract’s safety and security. Therefore, Guardian does not guarantee the explicit security of the audited smart contract, regardless of the verdict.

About Guardian

Founded in 2022 by DeFi experts, Guardian is a leading audit firm in the DeFi smart contract space. With every audit report, Guardian upholds best-in-class security while achieving our mission to relentlessly secure DeFi.

To learn more, visit https://guardianaudits.com

To view our audit portfolio, visit https://github.com/guardianaudits

To book an audit, message https://t.me/guardianaudits